Unmasking Sorting Routines: An Investigative Approach

Back in the day, students in data science courses were often tasked with organizing scrambled datasets, uncovering hidden structures through a process known as data sorting and analysis. In this task, your objective is to identify several unknown algorithms by analyzing their computational behaviors, using a similar investigative approach.

We provide you with a compiled library that contains five distinct sorting routines, labeled **MysterySort1**, **MysterySort2**, and so forth. Each of these routines is designed to take a vector of numbers and arrange the elements in ascending order. However, the key difference between them is that each routine implements a different sorting algorithm. The specific algorithms employed are (listed alphabetically):

1) **Bubble sort**

2) **Insertion sort**

3) **Merge sort**

4) **Quick sort** (first element of subvector is used as pivot)

5) **Selection sort**

Your task as a sort detective is to determine which sorting algorithm is implemented by each mystery sort function. Along with identifying the correct algorithm for each function, you must provide a detailed explanation of the evidence and reasoning you used to arrive at your conclusions.

You're largely free to design and conduct these experiments as you see fit. However, one key factor you should focus on is the timing of each algorithm, as it can provide crucial insights. For instance, the difference between an $O(N^2)$ algorithm and an O(N log N) algorithm should be evident in how their execution times scale with increasing input sizes. Additionally, the efficiency of some algorithms can vary depending on the initial arrangement of the elements in the input vector. Therefore, preparing different input configurations and comparing the resulting performance across the mystery sorts will likely yield valuable information for identifying each algorithm.

Note: The time for executing each sorting algorithm can be measured using ctime library in C++.

```
#include <ctime>

#include int main()
{
        double start = double(clock()) / CLOCKS_PER_SEC ;
       . . . Execute your sorting function . . .
       double finish = double(clock()) / CLOCKS_PER_SEC ;
       double elapsed = finish - start ;
}
```

Unfortunately, accurately measuring the time required for a program that executes quickly can be tricky, as the system clock may not be precise enough to capture the elapsed time. For instance, if you attempt to time the sorting of a small dataset, such as 10 integers, it's likely that the elapsed time recorded will be zero. This happens because modern processing units can execute many instructions within a single clock tick, potentially completing the entire sorting process before the clock advances. As a result, the recorded start and finish times might be identical, making it appear as though no time has passed.

To address this issue, one effective approach is to repeat the calculation multiple times between the two calls to the clock function. For instance, if you want to measure the time it takes to sort 10 numbers, you can conduct the sort-10-numbers experiment 1000 times consecutively, and then divide the total elapsed time by 1000. This method yields a much more accurate timing measurement. As part of your experimentation, you will need to determine how many repetitions are necessary for different input sizes to obtain reasonably precise results.

Our mystery sorting routines also include a feature that allows you to specify a maximum amount of time for the sort to run. By choosing an appropriate value, you can stop the sort mid-stream, and then use the debugger (or print statements) to examine the partially sorted data and try to determine the pattern with which the elements are being rearranged (e.g., Quicksort's pivot behavior, Selection Sort's partially sorted segments). Through a combination of these types of experiments, you should be able to figure out which algorithm is which and assert your conclusions with confidence.

Tasks

1) Map which sorting algorithm corresponds to which MysterySortx in the compiled library (can be created on your own needed for your analysis)

2) Print the time for all the algorithms (or by any other means of your wish).
3) Your analysis of each sorting algorithm over various inputs.

Input

1) Compiled library of all the aforementioned mystery sorting algorithms.
2) Multiple Initial configurations (a text file with integer numbers to be sorted).
3) Maximum amount of time for sorting (Same time for all algorithms).

Output

Identification of Sorting Algorithms

1) MysterySort1 = Bubble Sort
2) MysterySort2 = Quick Sort
3) MysterySort3 = Merge Sort
4) MysterySort4 = Selection Sort
5) MysterySort5 = Insertion Sort

Timing Results

MysterySort1 (Bubble Sort):

- 10 elements: 0.002 seconds
- 100 elements: 0.050 seconds
- 1000 elements: 5.200 seconds

MysterySort2 (Quick Sort):

- 10 elements: 0.001 seconds
- 100 elements: 0.005 seconds
- 1000 elements: 0.020 seconds

… etc ...

Format of analysis of Sorting algorithms expected during the evaluation by the TAs (can be explained orally).

Example

- MysterySort1 consistently exhibited quadratic time complexity ($O(N^2)$) as shown by the significant increase in execution time with larger inputs. The final pattern of sorting suggests a Bubble Sort, where elements "bubble" to their correct positions.

- MysterySort2 showed a divide-and-conquer strategy with logarithmic components, typical of Quick Sort, especially visible with the use of the first element as a pivot. The performance improved with nearly sorted inputs.

… so on

Partial Analysis

Partial Sorting Observations

- MysterySort3: When interrupted, the array showed that elements were being merged from two sub-arrays, confirming it as Merge Sort.

- MysterySort4: Early passes revealed the smallest elements were progressively moved to their correct positions, consistent with Selection Sort.

… so on ...

—---------- **All the best** —----------